

TEMA 1. GRAMATICAS Y LENGUAJES

- Concepto de gramática.
- Operaciones con lenguajes.
- Limpieza de gramáticas.

CONCEPTO DE GRAMÁTICA

1. Terminología básica

- **Alfabeto:** conjunto finito y no vacío de símbolos.
- **Símbolo:** elemento de un alfabeto.
- **Palabra:** concatenación finita de símbolos.
- **A^* :** conjunto de todas las palabras que se pueden formar con el alfabeto A .
- **Lenguaje:** subconjunto de palabras de un alfabeto.
- **Concatenación:** encadenamiento de dos palabras.
- **Cadena inversa:** inversión de los símbolos de una palabra.
- **Cadena iterada:** concatenación de una palabra consigo misma i veces.
- **Cadena vacía:** cadena formada por cero caracteres. La notaremos como ξ .
- **A^+ :** conjunto de todas las palabras que pueden formarse con el alfabeto A , exceptuando la cadena vacía.
- **Regla de producción:** $S \rightarrow aBA b$. Podemos sustituir el símbolo S por la cadena $aBA b$.

Ejemplo: Sea $A=\{a, b\}$. $L_1=\{a, b, \xi\} \subseteq A^*$

Ejemplo: Sea $A=\{a, b\}$. $L_1=\{a^i b^i / i=0, 1, \dots\}$

Ejemplo: Sea $A=\{a, b\}$. $L_1=\{uu^{-1} / u \in A^*\}$

2. Concepto de Gramática

Llamamos *Gramática Generativa* a la cuádrupla $G = (V, T, P, S)$ donde:

- V : alfabeto de *variables* o símbolos *no terminales*.
- T : alfabeto de símbolos *terminales*.
- P : conjunto de *reglas de producción*.
- S : Símbolo especial de V denominado *axioma* o *símbolo inicial* de la gramática.

Ejemplo: $G = (V, T, P, S)$ donde:

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow aB, S \rightarrow bA, A \rightarrow a, A \rightarrow aS, A \rightarrow bAA, B \rightarrow b, B \rightarrow bS, B \rightarrow aBB\}$$

Ejemplo: $G = (V, T, P, E)$ donde:

$$V = \{E\}$$

$$T = \{+, -, \cdot, (,), <id>\}$$

$$P = \{E \rightarrow E+E, E \rightarrow E-E, E \rightarrow E \cdot E, E \rightarrow (E), E \rightarrow <id>\}$$

3. Paso de derivación

Sea $G = (V, T, P, S)$ y $\alpha, \beta \in \{V \cup T\}^*$. Decimos que β se deriva de α en un paso y se nota como $\alpha \Rightarrow^+ \beta$ sí y solo sí:

$$\exists \gamma_1 \text{ y } \gamma_2 \in \{V \cup T\}^* \text{ y } \delta \rightarrow \varphi \in P / \alpha = \gamma_1 \delta \gamma_2 \text{ y } \beta = \gamma_1 \varphi \gamma_2$$

Sea $G = (V, T, P, S)$ y $\alpha, \beta \in \{V \cup T\}^*$. Decimos que β se deriva de α y se nota como $\alpha \Rightarrow^* \beta$ sí y solo sí:

$$\exists \gamma_1, \dots, \gamma_n \in \{V \cup T\}^* / \alpha = \gamma_1 \Rightarrow^+ \dots \Rightarrow^+ \gamma_n = \beta$$

4. Lenguaje generado por una gramática

Se define como el conjunto de cadenas compuestas por símbolos terminales que se derivan del axioma de la gramática dada:

$$L(G) = \{u \in T^* / S \Rightarrow^* u\}$$

Ejemplo: Obtener el lenguaje generado por $G = (\{S\}, \{a, b\}, \{S \rightarrow aSb, S \rightarrow ab\}, S)$.

5. Clasificación de las gramáticas según Chomsky

- **Tipo 0:** Es el tipo más general. Definen lenguajes con estructura de frases. Las *máquinas de Turing* reconocen lenguajes de este tipo.
- **Tipo 1:** *Gramáticas sensibles al contexto.* Los *autómatas lineales acotados* reconocen lenguajes de este tipo. Las producciones son de la forma:

$$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2 / A \in V, \beta \in (V \cup T)^+, \alpha_1, \alpha_2 \in (V \cup T)^*.$$

- **Tipo 2:** *Gramáticas libres de contexto.* Los *autómatas a pila* reconocen los lenguajes generados por estas gramáticas.

Las producciones son del tipo:

$$A \rightarrow \alpha / A \in V, \alpha \in (V \cup T)^*$$

- **Tipo 3:** *Gramáticas regulares.* Los lenguajes reconocidos por estas gramáticas se denominan

lenguajes regulares, y son reconocidos por *autómatas finitos*.

Las producciones pueden tener la siguiente forma:

$$A \rightarrow u, A \rightarrow uB \quad / \quad A, B \in V, \quad u \in T^*$$

Denominándose *gramática regular lineal por la derecha*.

O también:

$$A \rightarrow u, A \rightarrow Bu \quad / \quad A, B \in V, \quad u \in T^*$$

Denominándose en este caso *gramática regular lineal por la izquierda*.

6. Notación de Backus

Permite abreviar la especificación de la parte izquierda de las producciones de una gramática, siempre que haya al menos dos producciones cuyas partes izquierdas sean idénticas.

Ejemplo: Expresar en notación de Backus la siguiente gramática $G = (\{S, A, B\}, \{a, b\}, \{S \rightarrow aB, S \rightarrow bA, A \rightarrow a, B \rightarrow b, A \rightarrow aS, B \rightarrow bS\})$

7. Gramáticas ambiguas

Una gramática se dice que es **ambigua** si existe al menos una palabra del lenguaje generado por dicha gramática con al menos dos árboles de derivación.

OPERACIONES CON LENGUAJES

- **Unión:** $L_1 \cup L_2 = \{ u / u \in L_1 \text{ ó } u \in L_2 \}$
- **Intersección:** $L_1 \cap L_2 = \{ u / u \in L_1, u \in L_2 \}$
- **Diferencia:** $L_1 - L_2 = \{ u / u \in L_1, u \notin L_2 \}$
- **Complementario:** $L \subseteq A^* \rightarrow \sim L = A^* - L$
- **Concatenación:** $L_1 L_2 = \{ z=uv / u \in L_1, v \in L_2 \}$
- **Iteración:** $L^0 = \{\xi\}; L^{i+1} = L^i L$
- **Clausura de Kleene:** $L^* = L^0 \cup \dots \cup L^n; L^+ = L^* - \{\xi\}$
- **Simétrico o inverso de L:** $L^{-1} = \{u^{-1} / u \in L\}$
- **Cabecera de un lenguaje:** $CAB(L) = \{u \in A^* / \exists v \in A^* / uv \in L\}$
- **Homomorfismo entre alfabetos:** Si A_1 y A_2 son dos alfabetos, la aplicación h se dice que es un homomorfismo entre A_1^* y A_2^* si mantiene la concatenación de cadenas; es decir:

$$\begin{aligned} h: A_1^* &\rightarrow A_2^* \\ h(uv) &= h(u)h(v) \\ h(\xi) &= \xi \end{aligned}$$

- **Isomorfismo:** un homomorfismo es un isomorfismo si y sólo si:

$$h(u) = h(v) \Leftrightarrow u = v$$

LIMPIEZA DE GRAMATICAS

1. Reglas innecesarias

Son del tipo $A \rightarrow A$, y hacen que una gramática sea ambigua.

2. Símbolos inaccesibles

Son todos aquellos símbolos que no sean accesibles de forma directa o indirecta desde al menos una de las producciones asociadas al axioma.

3. Reglas superfluas

Si para un símbolo no terminal no existe una derivación a símbolo terminal, todas las reglas que contengan dicho símbolo son reglas superfluas.

4. Gramática reducida

Llamamos *gramática reducida* a aquella que no tiene símbolos inaccesibles ni superfluos.

5. Gramática limpia

Llamamos *gramática limpia* a aquella gramática reducida que carece de reglas innecesarias.

6. Algoritmo de limpieza de gramáticas

1. Eliminar directamente todas las **reglas innecesarias**.
2. Hacer
 - 2.1. Marcar las producciones para las que exista una regla $A \rightarrow x$ donde x sea una cadena de símbolos terminales o de no terminales situados en la parte izquierda de producciones previamente marcadas.
Hasta que no se marque ninguna producción.
3. Las reglas no marcadas son **reglas superfluas** y deberán eliminarse.
4. Sea H el cjto. de variables accesibles desde el axioma S .
Sea $\alpha, \beta \in (V \cup T)^*$ y sea $Y, Z \in V$
 - 4.1. $H = \emptyset$
 - 4.2. $\forall S \rightarrow \alpha$, si $Z \in \alpha$ y $Z \in V$ entonces $H = H \cup \{Z\}$
 - 4.3. Mientras H cambie hacer:
 - 4.3.1. $\forall Y \in H, \forall Y \rightarrow \beta$, si $Z \in \beta$ y $Z \in V$ entonces $H = H \cup \{Z\}$
 - 4.4. $\forall Y \rightarrow \alpha$ t.q. $Y \notin H$, eliminar dicha producción, por ser dicho **símbolo inaccesible**.

Ejemplo: Limpiar la gramática $G = (\{0, 1\}, \{A, B, C, S\}, S, P)$, donde P viene definido por las siguientes producciones:

$$\begin{aligned} S &\rightarrow 0S0 \mid 0B1 \mid S \mid A \mid \xi \\ A &\rightarrow 1A1 \mid \xi \\ B &\rightarrow 0B1 \\ C &\rightarrow 0B \mid A0B1 \mid 0 \mid 1 \mid \xi \end{aligned}$$